Detection of Human and LLM-Generated Text

Yujia Ming June 29, 2026

1 Introduction

1.1 Background

With the rapid development of large language models, machine-generated text has approached, and in some settings reached, human-level fluency, coherence, and linguistic quality. It has become increasingly difficult to tell whether a text was written by a person or generated by a model. Because large language models (LLMs) can generate text quickly and their output increasingly resembles human writing, people are using them more often and in more domains. This reduces production costs but also creates social risks. For example, LLMs may generate false information in news and accelerate the spread of rumors; students may use them to write papers, raising academic integrity concerns. Moreover, if AI-generated text is used to train the next generation of models, data quality may deteriorate and model performance may decline. Determining whether a given text was written by a human or an LLM, and, beyond that, which LLM generated it, has therefore become an important problem.

Existing research usually frames the detection of LLM-generated text as a binary classification task: deciding whether an input was written by a person or generated by a model. Methods generally fall into four groups: statistical and traditional machine learning methods, neural network methods, zero-shot and probabilistic methods, and watermarking methods (Wu et al., 2024). First, early statistical and traditional machine learning methods relied mainly on features such as word frequency, syntax, and perplexity (Beresneva, 2016; Jawahar et al., 2020), but their performance declines markedly on GPT-3.5 and newer models (Uchendu et al., 2021). Second, neural network methods typically fine-tune pretrained models such as BERT to learn differences between the distributions of human and machine text (Li et al., 2023; Liu et al., 2023). They perform well on in-distribution data but generalize less well across models or domains. Third, zero-shot and probabilistic methods do not require large labeled datasets. They exploit differences in perplexity or local curvature in the probability space of model-generated text. DetectGPT and Fast-DetectGPT are representative examples (Mitchell et al., 2023; Bao et al., 2023). This is one of the most active approaches in recent years. Fourth, watermarking methods embed implicit markers during generation so that later detection can examine token distributions or trigger patterns. Watermarking is more suited to engineered generation pipelines than to our task of training a detector on supplied data, but its underlying idea remains useful.

Existing datasets can be categorized by the model that generated the text and by application domain. Recent datasets use outputs from GPT-3.5, ChatGPT, GPT-4, LLaMA, and other LLMs, as in HC3, OpenLLMText, and DeepfakeTextDetect. Others combine multiple generators, as in M4 and DetectRL, to simulate real settings with several AI sources. Their domains include

question answering and knowledge (SQuAD and ELI5), news writing (GROVER), social media (TweepFake), and academic writing (CHEAT and ArguGPT). This report uses the competition dataset, which covers GPT, LLaMA, and Mistral, along with six text types: books, papers, news, question answering, social media, and reviews.

1.2 Project overview

This report extends binary detection to a multiclass task: it asks not only whether an input is human-written or model-generated, but also whether GPT, LLaMA, or Mistral generated it. The work has two main experiments. Experiment 1 follows the statistical and traditional machine learning approach. It constructs a ten-dimensional vector of statistical features and uses a multilayer perceptron (MLP) to predict labels 0, 1, 2, or 3 for the four-class task.

Experiment 2 takes a different approach. Because the literature has already studied binary detection extensively, our task can focus on identifying which model generated a text once the text is known to be AI-generated. Our dataset is also smaller than those used in many published papers. Experiment 2A therefore reproduces DetectGPT (Mitchell et al., 2023), using T5 to perturb text and measuring how the perturbations change its probability. Experiment 2B addresses the source-model question. Since LLMs all generate text by favoring higher-probability continuations, a direct extension of the zero-shot probabilistic approach may work poorly for distinguishing them. Inspired by watermarking, we instead hypothesize that differences in model architecture and training data create model-specific preferences for sets of tokens. Following watermark terminology, we call these sets Greenlists. We use statistical methods to recover each model's implicit Greenlist and use it to identify the source of a text.

In short, Experiment 1 adapts statistical and traditional machine learning methods, Experiment 2A builds on zero-shot probabilistic detection, and Experiment 2B draws on watermarking. The report thus implements three of the four major families of methods discussed above, providing an opportunity to compare their potential. Section 2 explains the methods; Section 3 reports their performance on the competition dataset and discusses possible reasons; Section 4 considers improvements; and Section 5 lists the references.

1.3 Experimental environment

The experiments used the web version of Google Colab as the main computing platform, with an A100 GPU to accelerate training and inference. The reported resources were 83.5 GB of system memory (about 4.7 GB in use), 40.0 GB of GPU memory (about 18.0 GB in use), and 235.7 GB of disk space (about 50.7 GB in use). The reported operating system was Windows. A terminal environment would be preferable if the browser connection is unstable.

The code used Python 3, primarily with PyTorch for model construction and training, NumPy and Pandas for data processing, and scikit-learn for data splitting and evaluation. Performance was assessed using accuracy, precision, recall, F1-score, and ROC-AUC. tqdm

displayed training progress, while standard Python libraries such as re and random supported text preprocessing and randomness control.

1.4 Experiment code

Experiment 1: https://colab.research.google.com/drive/1GMgoziK1pruFhtN04vHr7IMWzEOap9-g?usp=sharing

Experiment 2A: https://colab.research.google.com/drive/11_NeK64NKXo7p75ysbPQmtW0LBiI6jDZ?usp=sharing

Experiment 2B: https://colab.research.google.com/drive/1xMcSFYK_y1i55FeHQolqu8kvFYVunOZD?usp=sharing

Dataset: https://drive.google.com/drive/folders/1uhUck6hncp5dZoDHsOdaMgafGywKnKSK?usp=sharing

2 Methods

2.1 Experiment 1

Experiment 1 uses ten statistical features and an MLP to predict one of four labels. Let $l_t = \log p_\theta(x_t \mid x_{<t})$ denote the log probability of token x_t in its context. The features consist of five basic statistics of l_t, perplexity, two log-rank features, normalized probability entropy, and tail uncertainty. Table 1 gives their formulas and interpretations.

GPT-2 and GPT-2-large were used to calculate token log probabilities. Padding accelerated log-probability calculation, and MLP batch sizes of 64 or 128 accelerated training. The MLP has a 10-unit input layer for the extracted features, a 64-unit hidden layer with ReLU and 0.2 dropout, a 128-unit hidden layer with ReLU and 0.2 dropout, and a four-unit output layer for Human, GPT, LLaMA, and Mistral. Training uses softmax with cross-entropy loss.

2.2 Experiment 2A

Experiment 2A follows DetectGPT (Mitchell et al., 2023). Its premise is that model-generated text tends to lie near a local maximum in the generating model's probability space, whereas human-written text need not. If a passage is perturbed, for example by rewriting a sentence or masking and refilling a word, the model generally assigns the perturbed version a lower probability than the original model-generated passage. For human text, the probability may decrease, increase, or remain similar because the original need not be a local maximum. Averaging across perturbations therefore tends to cancel positive and negative changes for human text, while model-generated text tends to retain a larger gap.

Let $p_\theta(x)$ be the language model probability of input x, and draw a perturbed passage x'

Table 1: Text Features Extracted in Experiment 1

Category	**Feature**	**Formula**	**Interpretation**
Basic statistics	Mean log probability	$\mu = \frac{1}{T}\sum_{t=1}^{T} l_t$	Average token plausibility
	Variance	$\sigma^2 = \frac{1}{T}\sum_{t=1}^{T}(l_t - \mu)^2$	Variation in token confidence
	Minimum	$\min_t l_t$	Least certain token
	Maximum	$\max_t l_t$	Most certain token
	Median	$\text{median}(l_t)$	Robust central tendency
Perplexity	Perplexity	$\exp\left(-\frac{1}{T}\sum_{t=1}^{T} l_t\right)$	Overall generation difficulty
Log-rank	Mean log-rank	$\frac{1}{T}\sum_{t=1}^{T}\log(1 + r_t)$	Average token rank
	Rank variance	$\text{Var}(r_t)$	Dispersion in token ranks
Normalized entropy	Entropy	$-\sum_{t=1}^{T} p_t \log p_t$	Token distribution uncertainty
Tail uncertainty	Tail entropy	$-\sum_{t\in\mathcal{B}_{20\%}} p_t \log p_t$	Uncertainty in low-confidence tokens

Notes: T denotes the number of tokens; l_t is the log probability of token t; r_t denotes its rank; $p_t = e^{l_t}/\sum_j e^{l_j}$; and $\mathcal{B}_{20\%}$ denotes the bottom 20% of tokens ranked by confidence.

from $q(x' \mid x)$. The comparison is

$$\Delta(x) = \log p_\theta(x) - \mathbb{E}_{x' \sim q(x'|x)}[\log p_\theta(x')] \,.$$

With K perturbations, the empirical statistic is

$$S(x) = \log p_\theta(x) - \frac{1}{K} \sum_{k=1}^{K} \log p_\theta(x'_k).$$

Here x is the original passage, x'_k is its kth perturbed version, and K is the number of perturbations. Model-generated text is expected to have a larger S(x) than human text. A threshold on S(x) can thus support binary classification. We used GPT-2 to calculate token log probabilities and T5 to create perturbations, with batching for speed. We compared four and ten perturbations.

2.3 Experiment 2B

Experiment 2B adapts the idea of watermarking. We assume that differences in model architecture and training data may give each LLM a preference for certain tokens. This can help distinguish source models among texts already known to be AI-generated. The training set is divided into GPT, LLaMA, and Mistral subsets. For texts produced by model j, the observed count of token i is

$$N_i^{(j)} = \sum_{x \in D_j} \sum_t \mathbf{1}(x_t = i),$$

where D_j is the set of texts generated by model j and $\mathbf{1}(\cdot)$ is an indicator function. A separate language model k is then used as a reference. For a passage $x = (x_1, \ldots, x_T)$, it gives the next-token distribution $p_\theta(v \mid x_{<t})$ for vocabulary token v. To limit computation and accommodate the small sample, only the 30 highest-probability tokens are retained at each step. The expected count for token i under reference model k is estimated as

$$E_i^{(k)} = \sum_{x \in D_j} \sum_t p_\theta(i \mid x_{<t}),$$

where only top-30 predictions are counted.

A token whose observed count substantially exceeds the reference expectation may be preferred by the source model. We therefore compute a standardized residual:

$$z_i^{(j,k)} = \frac{N_i^{(j)} - E_i^{(k)}}{\sqrt{E_i^{(k)}}}.$$

This is approximately the form of a standardized Poisson residual. A positive z-score means that a token occurs more often than predicted by the reference model; a negative score means it

occurs less often. Larger values suggest stronger model-specificity. Tokens are ranked by z-score, and the top K tokens form model j's recovered Greenlist:

$$G_j = \{g_1, g_2, \ldots, g_K\}.$$

To measure how similar the three recovered token preferences are, we use Jaccard similarity for each pair of Greenlists:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Higher Jaccard similarity indicates more shared preferred tokens; lower similarity suggests more distinctive sets. We compare GPT with LLaMA, GPT with Mistral, and LLaMA with Mistral.

Membership in a Greenlist alone is insufficient for classification, so we also use each token's z-score as a weight. For a candidate text x, model j's Greenlist score is

$$S_j(x) = \sum_{g \in G_j} z_g f(g, x),$$

where f(g, x) is the frequency of token g in x and z_g is its Greenlist weight. Scores are computed for GPT, LLaMA, and Mistral; the highest-scoring model is predicted:

$$\hat{y} = \arg\max_j S_j(x).$$

3 Experimental results

3.1 Experiment 1

3.1.1 Training and validation performance

Table 2. Final six training epochs of the MLP

Epoch	Training loss	Validation accuracy	Validation F1
95	1.1950	0.5095	0.1688
96	1.1917	0.5095	0.1688
97	1.1911	0.5100	0.1689
98	1.1927	0.5100	0.1689
99	1.1934	0.5100	0.1689
100	1.1908	0.5100	0.1689

Using GPT-2-large to calculate contextual token log probabilities as an example, with

similar results for GPT-2, Table 2 shows that training loss stabilized after about 80 epochs at approximately 1.19. Validation accuracy remained around 51%, while macro-F1 was about 0.169. Additional epochs brought almost no improvement, suggesting that the MLP had largely converged. The performance limit therefore appears to reflect the small amount of discriminative information in these ten basic features, rather than insufficient training.

3.1.2 Test-set performance

We replaced GPT-2 with the larger GPT-2-large model for feature extraction to test whether model capacity was the bottleneck. Accuracy did not improve appreciably; it fell slightly from 53.6% to 53.5%. This makes the prospects of this approach for our multiclass task uncertain.

3.2 Experiment 2A

3.2.1 Four perturbations

The best classification threshold selected on the training set was 8.266. At this threshold, the false positive rate (FPR) was 5% and the true positive rate (TPR) was only 16.03%. Thus, 5% of genuinely human texts were incorrectly labeled as AI-generated, while only 16.03% of AI-generated texts were correctly detected. Test accuracy was 52.7%. On this binary task, DetectGPT performed similarly to, but slightly worse than, the MLP approach in Experiment 1.

To examine the weak binary performance, we plotted DetectGPT scores for human and AI text (Figure 1). The two distributions overlap substantially and show no clear separation, apart from a small long tail on the human side. Although perturbation produces some difference in log-probability changes, its discriminative power is limited. Four perturbations may be too few, so we next increased the count to ten.

3.2.2 Ten perturbations

With ten rather than four perturbations, the best threshold selected from the training set was 5.1413. The validation FPR was 4.49%, and the TPR rose to 21.64%. Test accuracy increased from 52.7% to 53.7%, reaching approximately the level of the MLP in Experiment 1. The higher TPR suggests that more perturbations estimate the local probability curvature more consistently, reducing random variation in the DetectGPT statistic and allowing some previously ambiguous AI text to be recognized. The FPR remained near 5%, so the increased perturbation count did not greatly increase misclassification of human text.

Figure 2 shows that AI text shifted toward higher DetectGPT scores, while human text remained concentrated at lower scores. Nevertheless, many samples from both groups still have similar scores and cannot be distinguished reliably. The reported improvements in TPR and test accuracy, together with Table 3, show the remaining limitations. In these experiments, DetectGPT's effectiveness appears unstable and leaves room for improvement.

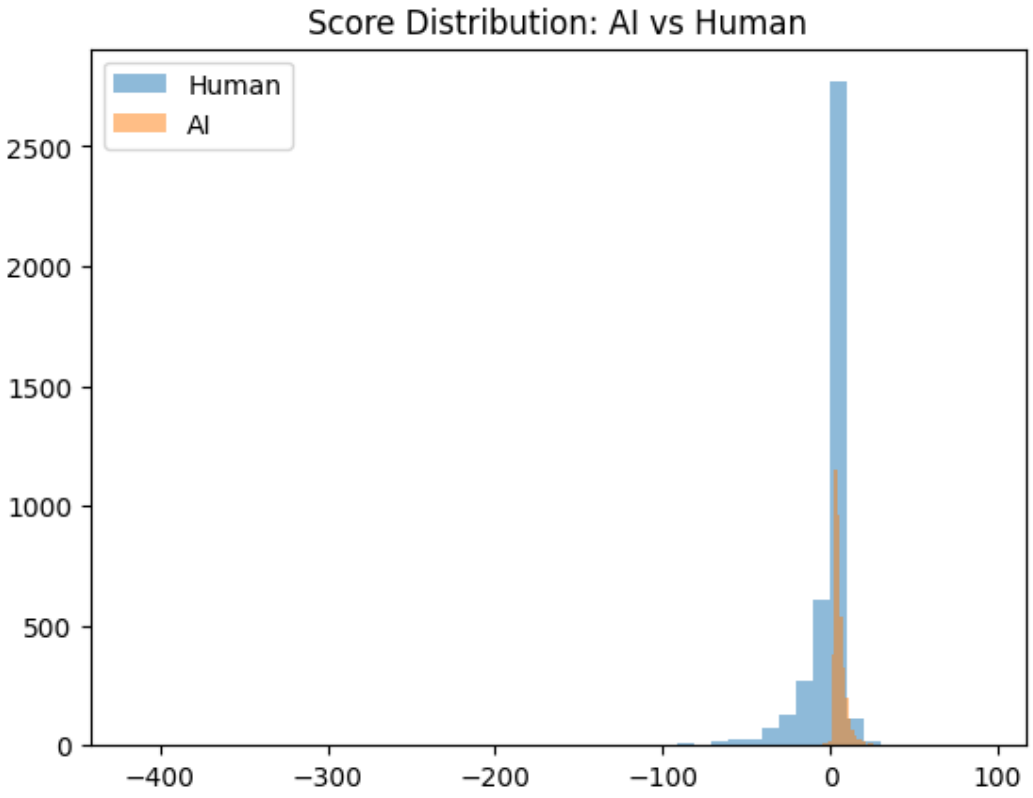


Figure 1: DetectGPT score distributions for human and AI text with four perturbations

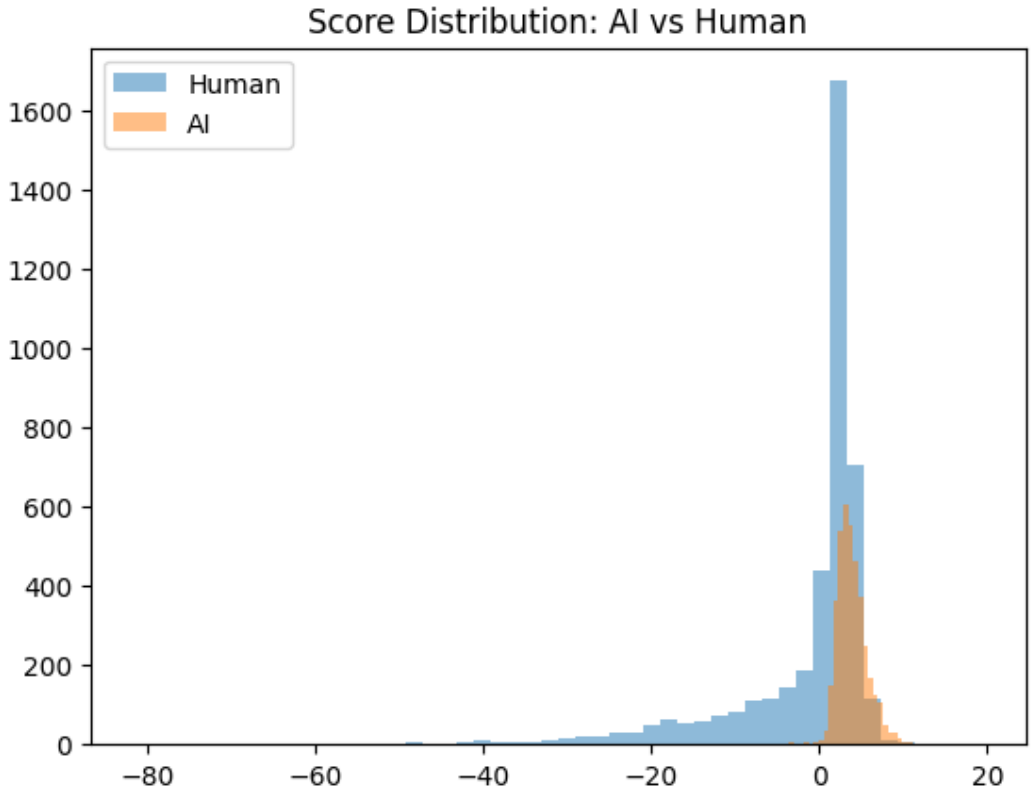


Figure 2: DetectGPT score distributions for human and AI text with ten perturbations

Table 3. Validation performance with ten perturbations

Class	Precision	Recall	F1-score	Support
0	0.5499	0.9392	0.6937	1020
1	0.7597	0.2000	0.3166	980
Accuracy	—	—	0.5770	2000
Macro avg	0.6548	0.5696	0.5052	2000
Weighted avg	0.6527	0.5770	0.5089	2000

3.3 Experiment 2B

3.3.1 Overlap among model Greenlists

Experiment 2B first recovered candidate Greenlists for GPT, LLaMA, and Mistral from the training set. Jaccard similarity was highest between GPT and Mistral (0.113), followed by LLaMA and Mistral (0.075), and lowest between GPT and LLaMA (0.037). All three values were below 0.12. This indicates limited overlap among the recovered high-preference token sets: most tokens had high statistical significance for only one model. The conclusion needs confirmation on a larger dataset. Tables 4–6 show the top ten Greenlist tokens learned from the competition dataset.

Table 4. Top ten GPT Greenlist tokens

Token ID	Token	N	E	Z-score
28878	. . .	323	51.501	37.832
1101	...	336	100.773	23.432
28745	;	501	214.857	19.521
5923	.\	395	155.734	19.173
1096	because	518	246.788	17.264
28705	(space)	3410	2557.239	16.863
1024	after	612	320.095	16.316
330	a	455	216.356	16.224
325	(	1583	1056.797	16.187
1040	—	142	40.191	16.059

Table 5. Top ten Mistral Greenlist tokens

Token ID	Token	N	E	Z-score
4100	important	927	428.503	24.082
5806	while	451	195.053	18.326

Token ID	Token	N	E	Z-score
19814	additionally	466	222.902	16.283
6716	one	322	144.290	14.794
8228	grade	100	33.926	11.344
4658	believe	534	335.125	10.864
512	in	672	444.493	10.791
4477	follows	107	40.038	10.582
3112	it	137	59.860	9.970
1667	main	137	59.865	9.969

Table 6. Top ten LLaMA Greenlist tokens

Token ID	Token	N	E	Z-score
4525	like	651	134.954	44.422
2208	super	312	47.406	38.429
818	in	493	146.643	28.602
1722	as	333	79.370	28.469
1537	but	375	104.036	26.566
3404	stuff	378	105.365	26.560
1119	they	479	180.489	22.220
460	can	2629	1717.738	21.987
0	!	1194	663.980	20.569
11	,	22162	19400.680	19.825

3.3.2 Validation and test performance

When the Greenlist classifier was applied directly to all samples, validation accuracy was only 14.7%, below the roughly 25% accuracy of random guessing in a four-class task. Table 7 shows that the model predicted nearly every text as one class: class 2 had high recall, while recall for the other three classes was near zero. Weighted matching against the tokens in the recovered Greenlists did not capture the complex differences among the generators.

There are two likely reasons. First, even though the Greenlists overlap little, they represent only a small set of locally preferred tokens and do not cover a text's overall generation characteristics. For short texts or texts containing few Greenlist tokens, a few occurrences can dominate the weighted score, making classification unstable. Second, perturbations to test text can greatly change the prediction. Greenlists are interpretable, but their stability must be established on larger datasets and under text perturbation. This remains a promising direction for further research.

Table 7. Classification report on the validation set

Class	Precision	Recall	F1-score	Support
0	0.00	0.00	0.00	1020
1	0.24	0.02	0.04	360
2	0.15	1.00	0.25	286
3	0.00	0.00	0.00	334
Accuracy	—	—	0.15	2000
Macro avg	0.10	0.26	0.07	2000
Weighted avg	0.06	0.15	0.04	2000

3.3.3 Combining Experiments 2A and 2B

Although the Greenlist method performed poorly when used directly for four-class classification, performance improved somewhat when DetectGPT supplied auxiliary information about which texts were LLM-generated. With four DetectGPT perturbations, adding Greenlists yielded 53.2% test accuracy; with ten perturbations, test accuracy was 54.9%. Both methods still need substantial improvement to achieve a more satisfactory test result.

4 Future improvements

4.1 How can more LLMs be distinguished?

This report tested only GPT, LLaMA, and Mistral. As the number of LLMs grows, direct multiclass classification will become harder. Two possible approaches are: first, classify models by their underlying architecture or principles, identifying broad groups before individual models, since only a limited number of mainstream architectures tend to dominate at a given time; second, study the observable preferences in model generation. By analogy with revealed preference theory in microeconomics and Arrow's (1959) work on rationalizability, examining preferences in pairs and triples of models might establish consistency and identifiability across models. Greenlists may therefore have further potential.

4.2 Can DetectGPT and Greenlists be unified to identify human text?

We believe this may be possible. An LLM could likewise predict tokens in human-written text, allowing statistical methods to identify a set of characteristic words. In this case, the set would be a human Blacklist rather than a Greenlist.

4.3 How can Greenlists remain stable on larger datasets and under perturbation?

First, the approach should be trained on larger datasets, including those described in the background section. A separate Greenlist for each LLM could be learned for each text type. The competition data also contain multiple languages; if each language has enough examples, a separate Greenlist should be constructed for each LLM in each language. For text perturbations, T5 might be used to perturb each text at least ten times before applying the remaining steps.

4.4 Can test accuracy be improved further?

In addition to the improvements above, newer and more widely used models could replace the relatively old GPT-2.

References

[1] Arrow, K. J. Rational Choice Functions and Orderings. Economica, 1959, 26(102): 121–127.

[2] Beresneva, D. Computer-Generated Text Detection: Literature Review. Proceedings of the Student Research Workshop at EACL, 2016: 72–80.

[3] Jawahar, G., Abdul-Mageed, M., and Lakshmanan, L. V. S. Automatic Detection of Machine Generated Text: A Critical Survey. Proceedings of COLING, 2020: 2296–2309.

[4] Uchendu, A., Le, T., Shu, K., et al. TuringBench: A Benchmark Environment for Turing Test in the Age of Neural Text Generation. Findings of EMNLP, 2021: 2001–2016.

[5] Liu, Y., Chen, Y., et al. ArguGPT: Evaluating, Understanding and Detecting Arguments Generated by Large Language Models. Findings of EMNLP, 2023.

[6] Li, Y., Wang, X., et al. DeepfakeTextDetect: Deepfake Text Detection in the Wild. Proceedings of ACL, 2023.

[7] Wu, X., Zhong, Y., et al. A Survey on LLM-Generated Text Detection. Computational Linguistics, 2024.